

Stripe Payment Integration Guide

Next.js Frontend — 3DS / OTP Verification & Checkout Session

Project: LifeChoice Backend | Date: June 2026 | Backend: Django REST Framework |
Payment Gateway: Stripe

Table of Contents

1. Overview — Two Approaches
2. Why Some Cards Need OTP / 3DS Verification
3. Stripe Test Cards — Which Ones Trigger 3DS
4. Approach 1 — PaymentIntent + Manual 3DS (V1 API)
 1. Backend API Contract
 2. Complete Next.js Component
 3. The 3DS Pop-up Flow
 4. Why Your Team's Pop-up Didn't Work
5. Approach 2 — Stripe Checkout Session (V2 API — Current)
 1. Backend API Contract
 2. Complete Next.js Component
 3. Handling Success & Cancel Pages
6. Webhook — How Access Is Granted After Payment
7. Side-by-Side Comparison
8. Recommendation for Your Frontend Team
9. Environment Variables Checklist

1. Overview — Two Approaches

Your backend currently has **two distinct payment APIs** built side-by-side. They reach the same business goal (enrolling a user) but work very differently on the frontend.

V1 – PaymentIntent + 3DS (Old)

- Frontend collects card details using Stripe.js Elements
- Tokenises card into a `payment_method_id`
- Sends it to `POST /api/enrollment/`
- If 3DS is required, backend returns `client_secret`
- Frontend must call `stripe.handleCardAction()` to open OTP pop-up
- Frontend team did NOT implement this step → OTP never appeared

V2 – Checkout Session (Current)

- Frontend sends only `micro_credential_id` or `degree_id`
- Backend creates a Stripe Checkout Session and returns a URL
- Frontend redirects user to Stripe's hosted checkout page
- Stripe handles card collection, currency, AND 3DS natively
- No 3DS pop-up code needed on frontend
- User is redirected back to `/success` or `/cancel`

Current Status

Both V1 and V2 APIs exist in the backend. V2 (Checkout Session) is the recommended path because Stripe handles 3DS automatically. V1 is still available if you need embedded card forms in your UI.

2. Why Some Cards Need OTP / 3DS Verification

3D Secure (3DS) is an extra authentication layer mandated by banks, especially under the European PSD2 regulation and increasingly globally. It protects cardholders from unauthorised charges.

How It Works Conceptually

- 1 User enters card details**
Stripe risk engine evaluates transaction amount, card type, IP country, merchant category code, and past fraud signals.
- 2 Stripe / Bank decides: needs extra auth?**
The issuing bank sends back a signal through Stripe: *proceed* or *challenge required*.
- 3 Challenge (OTP pop-up)**
If authentication is required, a modal from the bank appears. The cardholder enters an OTP sent via SMS/email or approves via their banking app.
- 4 Authentication result**
Stripe receives the authentication result. If approved, the charge proceeds. If denied, payment fails.

Which Cards Require 3DS?

Card Type	3DS Likelihood	Reason
European Visa/Mastercard	Always	PSD2 mandates Strong Customer Authentication (SCA) in EU/EEA
Indian RuPay/SBI/HDFC	Always	RBI mandates 3DS (AFA - Additional Factor of Authentication) for all online transactions
Cards from UK banks (post-Brexit)	Very High	UK retained PSD2 equivalent rules after Brexit
US Visa/Mastercard — high-value transaction	Sometimes	Fraud risk scoring triggers it — not always required

US Visa/Mastercard — low-value, trusted merchant	Rarely	Liability shift may exempt low-risk transactions
Amex	Sometimes	Uses SafeKey — depends on bank program enrolment
Prepaid / virtual cards	Often	Banks treat these as higher risk

Important for Your Product

Since LifeChoice is targeting a global audience (including India, UK, EU), you should expect a significant percentage of users to go through 3DS. If 3DS is not handled, those payments will

silently fail

with status `requires_action` and users won't understand why their card was declined.

3. Stripe Test Cards – Which Ones Trigger 3DS

Card Number	Behaviour	Use Case
4242 4242 4242 4242	Succeeds directly	Happy path – no 3DS
4000 0025 0000 3155	3DS Required	Always triggers 3DS – test your OTP pop-up flow
4000 0000 0000 9995	Insufficient funds	Test declined card error handling
4000 0082 6000 3178	3DS Supported, not required	Optional 3DS – Stripe may or may not trigger it
4000 0027 6000 3184	3DS Required, fails	User fails 3DS authentication
4000 0000 0000 0069	Expired card	Test expired card errors
4000 0000 0000 0127	Incorrect CVC	Test CVC validation errors

Use
any future expiry
(e.g., 12/34),
any 3-digit CVC
(e.g., 123), and
any postal code
(e.g., 10001) for all test cards above.

4. Approach 1 — PaymentIntent + Manual 3DS (V1 API)

This is the OLD approach your team struggled with.

The V1 API returns `client_secret` when 3DS is needed, but your frontend team did not call `stripe.handleCardAction()`. This is why the OTP pop-up never appeared. This section explains the correct implementation so you understand what was missing.

4.1 Backend API Contract

ENDPOINT

```
POST /api/enrollment/
Authorization: Bearer <jwt-token>
Content-Type: application/json

{
  "micro_credential_id": 5,
  "payment_method_id": "pm_xxxxxxxxxxxx"  <-- from Stripe.js
}
```

RESPONSE: PAYMENT SUCCEEDED (200)

```
{
  "success": true,
  "message": "Payment successful! Micro-credential purchased.",
  "enrollment": { ... enrollment data ... }
}
```

RESPONSE: 3DS REQUIRED (202 ACCEPTED)

```
{
  "success": true,
  "message": "Payment requires additional authentication",
  "payment_intent": {
    "id": "pi_3abc...",
    "status": "requires_action",
    "client_secret": "pi_3abc..._secret_xxx",  <-- USE THIS
    "next_action": {
```

```

    "type": "use_stripe_sdk",
    ...
  }
}
}

```

The Critical Step Your Team Missed

When the backend returns HTTP 202 with `status: "requires_action"`, the frontend MUST call `stripe.handleCardAction(client_secret)`. This opens the bank's 3DS pop-up. Without this call, the payment stays in `requires_action` state forever and the user is never charged.

4.2 Complete Next.js Component (V1 — PaymentIntent with 3DS)

First, install dependencies:

```
npm install @stripe/stripe-js @stripe/react-stripe-js
```

FILE: `COMPONENTS/PAYMENT/PAYMENTFORMV1.TSX`

```

'use client';

import { useState } from 'react';
import { loadStripe } from '@stripe/stripe-js';
import {
  Elements,
  CardElement,
  useStripe,
  useElements,
} from '@stripe/react-stripe-js';

// Load Stripe with your publishable key (fetched from backend /api/stripe/pkey)
const stripePromise = loadStripe(process.env.NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY);

// — Inner form component (must be inside <Elements>) —
function CheckoutForm({ microCredentialId }: { microCredentialId: number }) {
  const stripe = useStripe();
  const elements = useElements();

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState(false);

```

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!stripe || !elements) return;

  setLoading(true);
  setError(null);

  const cardElement = elements.getElement(CardElement);
  if (!cardElement) return;

  // — STEP 1: Tokenise the card into a PaymentMethod —————
  const { paymentMethod, error: pmError } = await stripe.createPaymentMethod({
    type: 'card',
    card: cardElement,
  });

  if (pmError) {
    setError(pmError.message ?? 'Card error');
    setLoading(false);
    return;
  }

  // — STEP 2: Send payment_method_id to your backend —————
  const token = localStorage.getItem('access_token'); // your JWT
  const res = await fetch('/api/enrollment/', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({
      micro_credential_id: microCredentialId,
      payment_method_id: paymentMethod.id,
    }),
  });

  const data = await res.json();

  // — STEP 3a: Direct success —————
  if (res.status === 201 || (data.success && !data.payment_intent)) {
    setSuccess(true);
    setLoading(false);
    return;
  }

  // — STEP 3b: 3DS / OTP Required (status 202) —————
  // This is the step that was MISSING in your previous implementation.

```



```

// The backend returned client_secret but the frontend never used it.
if (res.status === 202 && data.payment_intent?.client_secret) {
  const clientSecret = data.payment_intent.client_secret;

  // This call opens the bank's OTP / 3DS pop-up modal
  const { paymentIntent, error: actionError } =
    await stripe.handleCardAction(clientSecret);

  if (actionError) {
    // User cancelled 3DS, or bank rejected authentication
    setError(actionError.message ?? '3DS authentication failed');
    setLoading(false);
    return;
  }

  // — STEP 4: 3DS passed – confirm payment with backend —————
  // After handleCardAction succeeds, the PaymentIntent status is
  // 'requires_confirmation'. We confirm it via backend (or directly).
  // Option A: Confirm directly via Stripe (no extra backend round-trip)
  const { error: confirmError } = await stripe.confirmCardPayment(clientSecret);

  if (confirmError) {
    setError(confirmError.message ?? 'Payment confirmation failed');
  } else {
    // The webhook (payment_intent.succeeded) will grant access.
    // Show success to user immediately.
    setSuccess(true);
  }
} else {
  // Other errors from backend
  setError(data.message ?? 'Payment failed');
}

setLoading(false);
};

if (success) {
  return (
    <div style={{ textAlign: 'center', padding: '40px', color: '#24b47e' }}>
      <h2>Payment Successful!</h2>
      <p>You are now enrolled. Check your email for confirmation.</p>
    </div>
  );
}

return (
  <form onSubmit={handleSubmit} style={{ maxWidth: '420px', margin: '0 auto' }}>
    <h2>Complete Payment</h2>

```

```

    /* Stripe's CardElement renders card number, expiry, CVC in one field
    <div style={{
      border: '1px solid #d1d5db',
      borderRadius: '8px',
      padding: '14px',
      margin: '20px 0',
    }}>
      <CardElement options={{ hidePostalCode: false }} />
    </div>

    {error && (
      <div style={{ color: '#e63946', marginBottom: '12px', fontSize: '14px',
        {error}
      </div>
    )}

    <button
      type="submit"
      disabled={loading || !stripe}
      style={{
        width: '100%',
        padding: '14px',
        background: loading ? '#a0aec0' : '#6772e5',
        color: '#fff',
        border: 'none',
        borderRadius: '8px',
        fontSize: '16px',
        cursor: loading ? 'not-allowed' : 'pointer',
      }}
    >
      {loading ? 'Processing...' : 'Pay Now'}
    </button>

    <p style={{ fontSize: '12px', color: '#718096', marginTop: '12px', text-align: 'center' }}>
      Secured by Stripe. Your card details are never stored on our servers.
    </p>
  </form>
);
}

// — Outer wrapper (provides Stripe context) —
export default function PaymentFormV1({ microCredentialId }: { microCredentialId: string }) {
  return (
    <Elements stripe={stripePromise}>
      <CheckoutForm microCredentialId={microCredentialId} />
    </Elements>
  );
}

```

```
);  
}
```

4.3 The 3DS Pop-up Flow Explained

1 User fills card form & clicks Pay

`stripe.createPaymentMethod()` converts card details into a safe token
`pm_xxx`

2 Backend receives pm_xxx

Django calls `stripe.PaymentIntent.create(amount, currency, payment_method, confirm=True)`

3a Card does NOT need 3DS → status: succeeded

Backend grants access immediately and returns HTTP 201. Done.

3b Card NEEDS 3DS → status: requires_action

Backend returns HTTP 202 with `client_secret`. Frontend MUST act on this.

4 Frontend calls `stripe.handleCardAction(clientSecret)`

Stripe opens an iframe/pop-up with the bank's 3DS challenge page. User sees OTP input.

5 User completes OTP in bank pop-up

Bank sends authentication result back to Stripe. Stripe closes the pop-up.

6 Frontend calls `stripe.confirmCardPayment(clientSecret)`

Finalises the charge. Stripe triggers `payment_intent.succeeded` webhook.

7 Backend webhook grants access & sends email

`handle_payment_intent_succeeded()` updates transaction status to `completed`.

4.4 Why Your Team's Pop-up Didn't Work

What Happened	What Should Have Happened
Backend returned 202 with <code>client_secret</code>	Frontend should have called <code>stripe.handleCardAction(client_secret)</code>
Frontend treated 202 as an error or ignored it	Frontend should detect <code>status === 'requires_action'</code> and open 3DS pop-up

No OTP dialog appeared	The OTP dialog only appears when <code>handleCardAction()</code> is called with the secret
Payment stayed in pending state	After OTP, frontend must call <code>confirmCardPayment()</code> to finalise the charge
Cards requiring 3DS always failed silently	User sees bank's authentication pop-up and enters OTP to approve

5. Approach 2 — Stripe Checkout Session (V2 — Current API)

This is the CURRENT approach and the recommended one.

With Stripe Checkout, you don't write any card collection UI or 3DS handling code. Stripe's hosted page handles everything — card form, 3DS pop-up, currency conversion, receipt. Your frontend only needs to redirect the user.

5.1 Backend API Contract

MC ENROLLMENT ENDPOINT

```
POST /api/v2/enrollment/
Authorization: Bearer <jwt-token>
Content-Type: application/json

{ "micro_credential_id": 5 }
```

— Response (200 OK) —

```
{
  "success": true,
  "message": "Checkout session created. Redirect user to checkout_url.",
  "checkout": {
    "checkout_url": "https://checkout.stripe.com/pay/cs_test_...",
    "session_id": "cs_test_..."
  }
}
```

DEGREE ENROLLMENT ENDPOINT

```
POST /api/v2/degree-enrollment/
Authorization: Bearer <jwt-token>
Content-Type: application/json

{ "degree_id": 3 }
```

— Response (200 OK) —

```
{
  "success": true,
  "checkout": {
    "checkout_url": "https://checkout.stripe.com/pay/cs_test_...",

```

```

    "session_id": "cs_test_..."
  }
}

```

5.2 Complete Next.js Component (V2 – Checkout Session)

No `@stripe/react-stripe-js` required for this approach. Only a simple redirect.

FILE: `COMPONENTS/PAYMENT/CHECKOUTBUTTON.TSX`

```

'use client';

import { useState } from 'react';

interface Props {
  type: 'mc' | 'degree';
  itemId: number;
  itemName: string;
  price: string;
}

export default function CheckoutButton({ type, itemId, itemName, price }: Props) {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handleCheckout = async () => {
    setLoading(true);
    setError(null);

    const token = localStorage.getItem('access_token');
    const endpoint =
      type === 'mc' ? '/api/v2/enrollment/' : '/api/v2/degree-enrollment/';
    const body =
      type === 'mc'
        ? { micro_credential_id: itemId }
        : { degree_id: itemId };

    try {
      const res = await fetch(endpoint, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          Authorization: `Bearer ${token}`,
        },
        body: JSON.stringify(body),
      });
    }
  };

```

```

const data = await res.json();

if (!res.ok || !data.success) {
  throw new Error(data.message ?? 'Failed to create checkout session')
}

// Redirect to Stripe's hosted checkout page.
// Stripe handles card entry AND 3DS OTP automatically on their page.
window.location.href = data.checkout.checkout_url;

} catch (err: any) {
  setError(err.message);
  setLoading(false);
}
};

return (
  <div>
    {error && (
      <div style={{
        background: '#fff0f0', border: '1px solid #fed7d7',
        color: '#c53030', padding: '12px 16px', borderRadius: '8px',
        marginBottom: '12px', fontSize: '14px'
      }}>
        {error}
      </div>
    )}

    <button
      onClick={handleCheckout}
      disabled={loading}
      style={{
        padding: '14px 28px',
        background: loading ? '#a0aec0' : '#6772e5',
        color: '#fff',
        border: 'none',
        borderRadius: '8px',
        fontSize: '16px',
        fontWeight: 600,
        cursor: loading ? 'not-allowed' : 'pointer',
        width: '100%',
      }}
    >
      {loading ? 'Preparing checkout...' : `Enroll in ${itemName} - ${price}`}
    </button>

    <p style={{ fontSize: '12px', color: '#718096', marginTop: '8px', text

```

```

        You will be redirected to Stripe's secure checkout page.
        3DS / OTP verification handled automatically.
    </p>
</div>
);
}

```

USAGE IN A PAGE

```

// app/courses/[id]/page.tsx

import CheckoutButton from '@components/payment/CheckoutButton';

export default function CoursePage({ params }: { params: { id: string } }) {
  return (
    <div>
      <h1>Micro-Credential: Advanced AI</h1>
      <CheckoutButton
        type="mc"
        itemId={parseInt(params.id)}
        itemName="Advanced AI"
        price="$14.99"
      />
    </div>
  );
}

```

5.3 Handling the Success and Cancel Pages

After Stripe completes (or cancels) payment, it redirects to your `success_url` or `cancel_url`. These are set on the backend in `settings.py`:

```

PAYMENT_SUCCESS_URL = "https://yourapp.com/enrollment/success"
PAYMENT_CANCEL_URL  = "https://yourapp.com/enrollment/cancel"

```

FILE: `APP/ENROLLMENT/SUCCESS/PAGE.TSX`

```

'use client';

import { useSearchParams } from 'next/navigation';
import { useEffect, useState } from 'react';

export default function PaymentSuccessPage() {

```



```

const params = useSearchParams();
const sessionId = params.get('session_id');
const type = params.get('type'); // 'mc' or 'degree'

return (
  <div style={{ textAlign: 'center', padding: '80px 20px', maxWidth: '480px' }}>
    <div style={{ fontSize: '56px', marginBottom: '20px' }}>✓</div>
    <h1 style={{ color: '#24b47e', marginBottom: '12px' }}>Payment Successful
    <p style={{ color: '#4a5568', marginBottom: '24px' }}>
      Your {type === 'degree' ? 'degree pathway' : 'micro-credential'} account is now
      activated. A confirmation email has been sent to your inbox.
    </p>

    {sessionId && (
      <p style={{ fontSize: '12px', color: '#a0aec0' }}>
        Reference: {sessionId}
      </p>
    )}

    <a href="/dashboard" style={{
      display: 'inline-block', marginTop: '24px',
      padding: '12px 28px', background: '#6772e5',
      color: 'fff', borderRadius: '8px', textDecoration: 'none',
      fontWeight: 600,
    }}>
      Go to Dashboard
    </a>
  </div>
);
}

```

FILE: APP/ENROLLMENT/CANCEL/PAGE.TSX

```

'use client';

import { useSearchParams } from 'next/navigation';

export default function PaymentCancelPage() {
  const params = useSearchParams();
  const type = params.get('type');

  return (
    <div style={{ textAlign: 'center', padding: '80px 20px', maxWidth: '480px' }}>
      <div style={{ fontSize: '56px', marginBottom: '20px' }}>✗</div>
      <h1 style={{ color: '#e63946', marginBottom: '12px' }}>Payment Canceled
      <p style={{ color: '#4a5568', marginBottom: '24px' }}>

```

```

    No charge was made. You can try again whenever you're ready.
  </p>

  <a href={type === 'degree' ? '/degrees' : '/courses'} style={{
    display: 'inline-block', marginTop: '24px',
    padding: '12px 28px', background: '#6772e5',
    color: 'fff', borderRadius: '8px', textDecoration: 'none',
    fontWeight: 600,
  }}>
    Back to {type === 'degree' ? 'Degrees' : 'Courses'}
  </a>
</div>
);
}

```

Important: Access is granted by the Webhook, not the Success Page

The success page is shown immediately when Stripe redirects, but access to the course is granted when the backend receives the `checkout.session.completed` webhook event. There may be a 1–3 second delay. Show a "Activating your access..." spinner on the success page if needed, and poll the enrollment API.

OPTIONAL: POLL FOR ENROLLMENT ACTIVATION ON SUCCESS PAGE

```

'use client';

import { useEffect, useState } from 'react';
import { useSearchParams } from 'next/navigation';

export default function PaymentSuccessPage() {
  const params = useSearchParams();
  const [enrolled, setEnrolled] = useState(false);

  useEffect(() => {
    // Poll enrollment API every 2 seconds until access is confirmed
    const interval = setInterval(async () => {
      const token = localStorage.getItem('access_token');
      const res = await fetch('/api/v2/enrollment/', {
        headers: { Authorization: `Bearer ${token}` },
      });
      const data = await res.json();
      if (data.enrollments?.length > 0) {
        setEnrolled(true);
        clearInterval(interval);
      }
    }, 2000);
  });
}

```

```
    }, 2000);

    // Stop polling after 30 seconds regardless
    setTimeout(() => clearInterval(interval), 30000);
    return () => clearInterval(interval);
  }, []);

  return (
    <div>
      {enrolled ? (
        <p>✔ Access activated! <a href="/dashboard">Go to Dashboard</a></p>
      ) : (
        <p>⌛ Activating your access... please wait.</p>
      )}
    </div>
  );
}
```

6. Webhook – How Access Is Granted After Payment

The webhook is the critical backend piece that connects a successful Stripe payment to actual course access. Your frontend does NOT call the webhook – Stripe calls it automatically.

- 1 Stripe Checkout page: user completes payment + 3DS**
Stripe processes the card, handles 3DS authentication internally, charges the card.
- 2 Stripe sends POST to your webhook endpoint**
`POST https://yourbackend.com/api/stripe/webhook/` with event type `checkout.session.completed`
- 3 Backend verifies webhook signature**
`stripe.Webhook.construct_event(payload, sig_header, webhook_secret)` – prevents fake requests
- 4 Backend reads metadata from session**
`user_id`, `micro_credential_id` or `degree_id`, `transaction_type` – set when session was created
- 5 Backend creates PaymentTransaction + grants MCAccess / DegreeAccess**
Atomic DB transaction ensures no partial states. Duplicate protection via session ID check.
- 6 Backend sends confirmation email to user & admin**
EmailService sends enrollment confirmation + admin notification asynchronously.
- 7 User's success page polls enrollment API**
Frontend detects the new enrollment and shows the "Access Activated" confirmation.

Webhook Endpoint Configuration

In your Stripe Dashboard (or via CLI), configure the webhook to point to:

`https://yourbackend.com/api/stripe/webhook/`

Listen for events: `checkout.session.completed`, `payment_intent.succeeded`, `payment_intent.payment_failed`, `charge.succeeded`, `charge.failed`

7. Side-by-Side Comparison

Feature	V1 – PaymentIntent (Old)	V2 – Checkout Session (Current)
Frontend card collection	Your custom form with Stripe Elements	Stripe's hosted page – zero frontend code
3DS handling	You must call <code>stripe.handleCardAction()</code>	Stripe handles automatically – no code needed
Frontend complexity	High – must handle <code>requires_action</code> state	Very low – just redirect to a URL
UI customisation	Full control over card form design	Limited to Stripe's checkout theme colours/logo
Currency conversion	Manual – backend must specify currency	Automatic (Adaptive Pricing on Stripe Dashboard)
PCI compliance	SAQ A-EP level – card data never touches your server but iframe must be correct	SAQ A level – simplest; user never leaves Stripe's domain for payment
Conversion rate	Can be higher if UX is seamless	High – Stripe's checkout is optimised for conversions
Mobile support	Requires careful iframe / pop-up handling	Native – Stripe's page is fully mobile-optimised
Apple Pay / Google Pay	Requires extra integration	Built-in – Stripe shows them automatically
Backend endpoint	<code>POST /api/enrollment/</code>	<code>POST /api/v2/enrollment/</code> or <code>/api/v2/degree-enrollment/</code>
Required request fields	<code>micro_credential_id</code> + <code>payment_method_id</code>	<code>micro_credential_id</code> only (or <code>degree_id</code>)
Access granting	Immediate (if no 3DS) or via <code>payment_intent.succeeded</code> webhook	Via <code>checkout.session.completed</code> webhook (1–3 sec delay)



8. Recommendation for Your Frontend Team

Use V2 (Checkout Session) as your primary integration

It requires ~10 lines of frontend code, handles 3DS completely automatically, and works with every card type worldwide including cards from India, EU, and UK that always require 3DS. Your previous 3DS pop-up issue is eliminated because Stripe handles it on their own hosted page.

Minimum Frontend Implementation Checklist (V2)

1. Create a "Enroll Now" button component that calls `POST /api/v2/enrollment/`
2. On API success, redirect to `data.checkout.checkout_url` using `window.location.href`
3. Create `/enrollment/success` page — shown after successful payment
4. Create `/enrollment/cancel` page — shown if user clicks "Back" on Stripe
5. (Optional) Poll enrollment list on success page to show "access activated" confirmation
6. Set `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` in your `.env.local`

If You Want an Embedded Card Form (V1) — Fix Checklist

1. Install `@stripe/stripe-js` and `@stripe/react-stripe-js`
2. Wrap your payment page in `<Elements stripe={stripePromise}>`
3. Use `CardElement` or `CardNumberElement` for card input
4. On submit: call `stripe.createPaymentMethod()` to get `pm_xxx`
5. POST `{ micro_credential_id, payment_method_id: pm_xxx }` to backend
6. If HTTP 202: extract `client_secret`, call `stripe.handleCardAction(client_secret)`
7. After `handleCardAction` resolves: call `stripe.confirmCardPayment(client_secret)`
8. Show success when `confirmCardPayment` returns without error

Never store card numbers

Never log, display, or transmit raw card numbers. Stripe.js ensures card data only flows to Stripe's servers. Your backend only ever sees the tokenised `pm_xxx` or `pi_xxx` IDs.

9. Environment Variables Checklist

Next.js Frontend (.env.local)

```
# Stripe publishable key — safe to expose in browser
# Fetch from GET /api/stripe/public-key/ at runtime, or hardcode here:
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_51...

# Your backend API base URL
NEXT_PUBLIC_API_BASE_URL=https://yourbackend.com
```

Django Backend (.env / settings.py)

```
# Set via Stripe dashboard or admin panel in the app:
STRIPE_SECRET_KEY=sk_test_51...          # Never expose this to frontend
STRIPE_WEBHOOK_SECRET=whsec_...          # From Stripe webhook configuration

# These must match your Next.js page routes:
PAYMENT_SUCCESS_URL=https://yourapp.com/enrollment/success
PAYMENT_CANCEL_URL=https://yourapp.com/enrollment/cancel
```

Stripe Dashboard Settings Required

Setting	Value	Where
Webhook Endpoint	<code>https://yourbackend.com/api/stripe/webhook/</code>	Stripe Dashboard → Developers → Webhooks
Webhook Events	<code>checkout.session.completed</code> , <code>payment_intent.succeeded</code> , <code>payment_intent.payment_failed</code>	Same webhook page — select events to listen to
Adaptive Pricing	Enable for automatic currency conversion	Stripe Dashboard → Settings →

		Checkout and Payment Links
3D Secure	No manual configuration needed — Stripe auto-handles based on card issuer	Automatic

Testing End-to-End

1. Use card `4242 4242 4242 4242` — should succeed without 3DS
2. Use card `4000 0025 0000 3155` — should trigger 3DS on Stripe's page (V2) or your `handleCardAction` modal (V1)
3. Complete 3DS by clicking "Complete" on the Stripe test 3DS page
4. Verify `checkout.session.completed` webhook fires in Stripe Dashboard → Events
5. Check that `MCAccess` or `DegreeAccess` record was created in Django admin
6. Verify enrollment confirmation email was received